

Two-Level HPS: a practical algorithm guide

Joseph Kump¹, Anna Yesypenko², and Per-Gunnar Martinsson¹

¹University of Texas at Austin

²Ohio State University



Keywords: Keyword1, Keyword2, Keyword3

INTRODUCTION

This technical guide is written as supplemental material to “A Two-Level Direct Solver for the Hierarchical Poincaré-Steklov Method”, by Kump et al. (2025). A literature review, mathematical justification, and numerical examples of our two-level HPS solver can be found in that manuscript. Here we provide a general guide to the organization and structure of the repository and how the command-line interface is used. We also discuss details regarding data structures, memory management, and CPU-GPU interaction within our code.

OVERVIEW OF THE CODE AND USE

A class/file diagram of the two-level HPS codebase can be seen in Figure 1. In summary, the repository has five layers of components, each corresponding to one row in the diagram:

1. **Entry layer:** The three driver modules (`argparse_driver`, `driver_build`, `driver_solve`) are the user-facing entry points that parse arguments, configure the PDE/domain, and run the solve pipeline respectively. `built_in_funcs` is a pure function library supplying manufactured solutions, boundary conditions, and coefficient fields.
2. **Abstract layer:** `AbstractHPSSolver` defines the interface contract (Schur complement blocks $\mathbf{A}_{ii}/\mathbf{A}_{ix}/\mathbf{A}_{xx}/\mathbf{A}_{xi}$, node index arrays $\mathbf{J}_i/\mathbf{J}_x$, solve utilities). `AbstractGeometry` enforces the bounds property. `PDO_2d` and `PDO_3d` are simple data-holders for PDE coefficients.
3. **Concrete implementations:** `Domain_Driver` is the central class, inheriting `AbstractHPSSolver` and orchestrating the build and solve stages. `BoxGeometry` and `ParametrizedGeometry2D/3D` are the two concrete geometry implementations extending `AbstractGeometry`, the latter adding forward/inverse parameter maps and transformed PDE coefficients for nonrectangular domains.
4. **Discretization & backends:** `HPS_Multidomain` owns the multi-box DtN assembly and leaf solve scheduling, calling into `hps_parallel_leaf_ops` for all GPU tensor work. `SparseSolver` abstracts the solver backends (MUMPS via `python-mumps` or PETSc, fallback to SciPy SuperLU).
5. **Leaf level:** `HPS_Disc` computes the single-leaf Chebyshev quadrature, differentiation matrices, and index arrays (the JJ namedtuples used to isolate leaf boundaries, corners, etc.). `CSRBuilder` is a utility for incrementally accumulating COO triplets before finalizing to CSR.

If using this solver standalone, then `argparse_driver` is the only file one needs to interact with directly. It allows the user to specify the nature of the PDE they wish to solve (differential operator, domain geometry and dimension, body loads and boundary conditions) as well as the nature of the discretization (choice of polynomial order p and degrees of freedom n) and solver architecture (CPU or GPU backend, application of static condensation, choice of sparse solver). The file `README.md` present in the GitHub repository shows example calls with these command line argument. Table 1 describes a list of potential arguments.

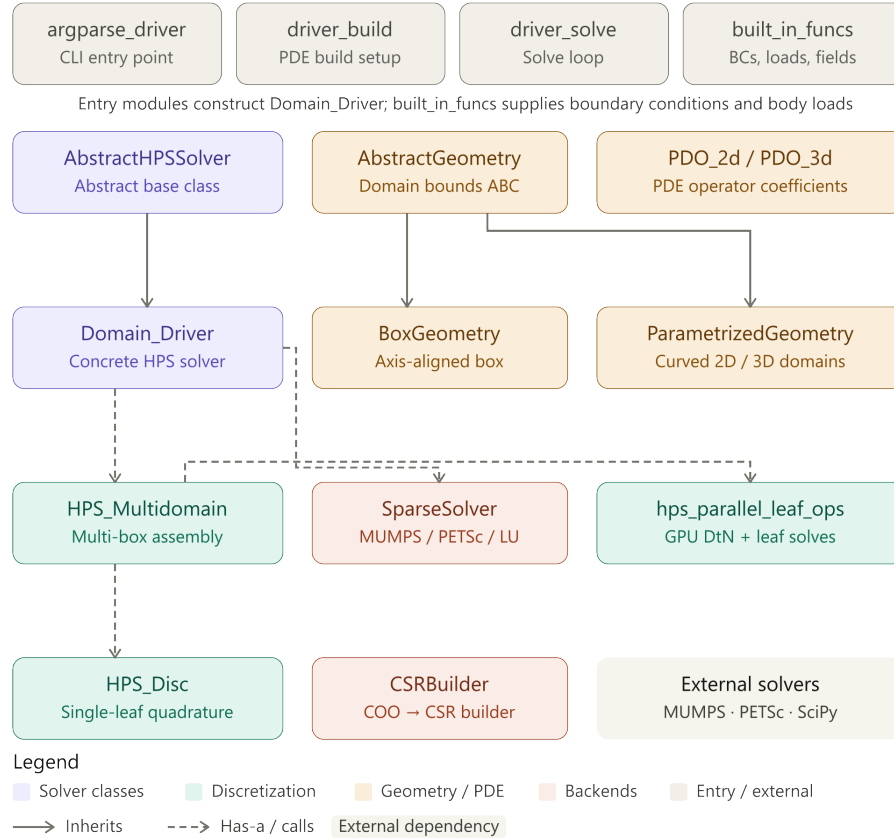


Figure 1. A diagram of the files and classes present in the two-level HPS formulation.

The two-level solver's code is also designed to interface with other PDE solvers, such as the one present in the GitHub repository `SslabLU`. The abstract classes present in `AbstractHPSSolver` and `AbstractGeometry` provide generalizations that can help this code interface with such packages.

DATA STRUCTURES

A pde solver of this kind necessarily must store several distinct components relevant to the problem:

1. Boundary data and body loads
2. Solution Arrays
3. Geometric Information
4. Differential Operators
5. Sparse Matrices

Boundary data and body loads include several arrays: `xx_active` holds coordinates for all leaf boundaries, with indexing vectors to differentiate between the global boundary and leaf boundaries interior to the global domain. Dirichlet data and body loads can be passed as either dense vectors or (if an analytic function exists) functions that are called on `xx_active` as needed. Indexing arrays `I_Ctot` and `I_Xtot` store the entries in `xx_active` corresponding to leaf boundaries on the domain interior (shared interfaces between leaves) and leaf boundareis on the global boundary, respectively.

Solution arrays include the solution restricted to `xx_active` (the solution on leaf boundaries) as well as the total solution computed as a tensor of shape $n_{\text{boxes}} \times p^d$, where p and d are the discretization's polynomial order and dimension, respectively. This array is flattened when returned after calling `domain_driver.solve()`. We also have an array storing all grid coordinates, `grid_xx`, of shape $n_{\text{boxes}} \times p^d \times d$.

Geometric information includes the aforementioned `grid_xx` for all coordinates and `xx_active` for grid coordinates restricted to leaf boundaries. If Gaussian interpolation is required, `xx_active` will store Gaussian nodes on the leaf boundaries instead of the Chebyshev nodes used for interior leaf solves. For supported nonrectangular arrays (sinusoidal curve, annulus, twisted torus), parameter maps are stored as functions and applied as needed when evaluating boundary conditions or differential operator coefficients. The solver also includes a reference Chebyshev node grid for one leaf, thus with p^d entries. A set of indexing arrays are also stored in the named tuples `JJ_2D`, `JJ_3D` to quickly grab data connected to certain regions (leaf boundaries, corners, etc.) from the total solution or body load.

Differential operators include the “building block” differential operators built from Kronecker products of 1D Chebyshev differentiation matrices. These are permanently stored in memory for the reference Chebyshev grid. The localized differential operators, including **A**, leaf solution operators **S**, and leaf Neumann operators **N** are not stored in memory since they may vary for each leaf, but are rather computed on-the-fly (directly on the GPU if one is in use) as needed for assembly and solves. If the differential operator has mixed second-order terms (usually induced by nonrectangular parameter maps), then Gaussian interpolation is used and interpolation matrices are also stored for the reference leaf grid.

Lastly, *sparse matrices* are stored in memory as SciPy sparse CSR matrices. These include `A_CC`, `A_CX`, `A_XC`, and `A_XX`, which store the different portions of DtN maps representing interactions between `I_Ctot` and `I_Xtot`. The most important one for our use is `A_CC`, which is equivalent to the reduced sparse system **T** as described in our main manuscript. `A_CC/T` is then factorized into a direct LU-solver through MUMPS or SuperLU and is stored as those solvers' relevant data types (such as `PETSc.ksp` for PETSc MUMPS). In theory we could use `A_XX` and `A_CX` in computing the reduced body load in the solve stage, but in practice since the body load reduction is dense it is better to reformulate and apply operators on the GPU instead.

MEMORY MANAGEMENT

All of the components described in the previous section are permanently stored on the CPU, with leaf operators assembled on the GPU as needed and relevant portions of boundary conditions, body loads, and solution data being sent to GPU as needed. Our current tests have not shown any errors related to running out of memory on the CPU. If CPU memory is a concern, however, then all of the sparse matrices except for `A_CC/T` can be deleted, and `A_CC` can be deleted as well once it has been factorized.

Memory is far more limited and thus a greater concern on the GPU. All GPU-based operations are limited to the file `hps_parallel_leaf_ops.py`, and carried out by the function `build_DtNs` (despite its name, this function also handles leaf interior solves and reduction of the body load for static condensation, since all three use the same batched linear algebra paradigm).

The two-level HPS solver was originally tested on an Nvidia V100 GPU, and written to be usable on modest GPU resources. Leaf-level operations including both DtN assembly and interior leaf solves require solving a system made from differentiable operators on the matrix interior, **A_{ii}**. Since each leaf in 3D has $(p-2)^3$ interior points, this matrix is of size $\approx p^6$, which is not trivially small. Even if the GPU can store the entire solution vector (the result of leaf interior solves, size = # discretization points, N) or all DtN maps (result of DtN assembly for sparse matrices, 3D size $\approx 6p^2 \times \#$ of leaves) concurrently, the temporary storage of the interior differential operators is too costly to store them all on the GPU for larger problem sizes.

To address this, our code features a two-layer batching scheme determining both the total amount of result stored on the GPU at one time (DtN maps, reduced body load, PDE solution) and the total number of operators that can be stored and used on the GPU at once, which is a smaller number. We define both of these values in the GPU-computation function `build_DtNs`, under the variable names `chunk_size` and `Aloc_chunk`, respectively. These can be viewed as outer and inner loops (or outer and inner “chunks”) of data transfers and GPU solves. A diagram of this approach is shown in Figure 2.

One GB of GPU memory is dedicated to the “outer” chunk size transferred between CPU and GPU. Dividing this by the memory size of a DtN map (in the build stage) or solution on a leaf interior (in the

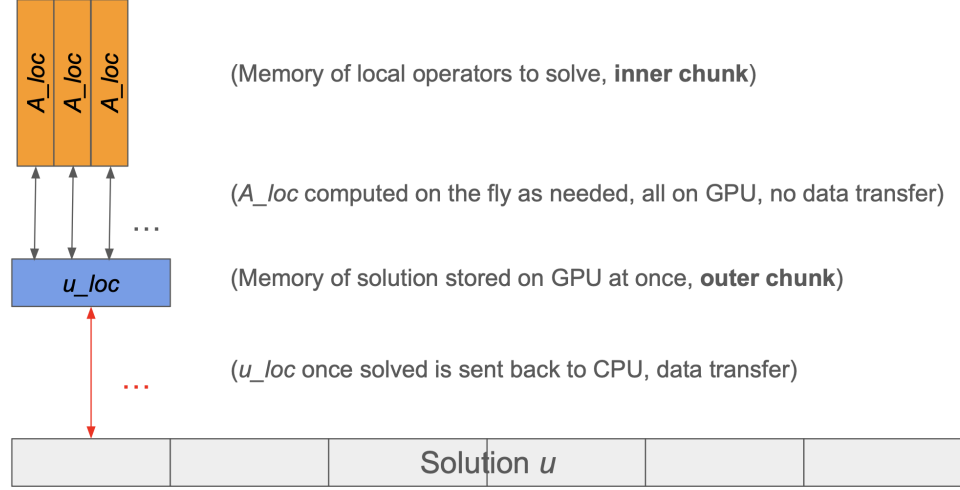


Figure 2. A diagram of the two-layer scheduler we use to manage memory on the GPU device, in this case for the solve stage. Data that exists on the CPU only is gray, data exchanged between CPU and GPU is blue, and data purely allocated and formulated on the GPU is orange. The inner loop of on-the-fly GPU allocations and solves is represented by black arrows, while the outer loop of transferring solution data between GPU and CPU is represented by red arrows.

solve stage) determines how many leaves’ worth of DtNs or solution operators can be stored on the GPU at once. To determine the viable size of these “inner” chunks, we use a conservative estimate based on the expected sizes of localized differential operators utilized in DtN formulation and leaf interior solves. This estimate is $4p^4 \times 8$ bytes for 2D problems, and $\approx (2p^6 + 12p^5 + 72p^4) \times 8$ bytes for 3D problems. We find the available reserved memory on the GPU and divide it by this estimate to determine how many leaves can be computed concurrently in one inner chunk. In practice these upper bounds for leaf operator size do not cause memory errors on a V100, though for higher p ($p > 16$) they typically lead to an inner chunk size of 1. Experimenting with refining the upper limits for both outer and inner memory chunks may lead to further improved performance, especially on newer-generation hardware like the Nvidia H100 GPU.

CPU-GPU INTERACTION

As briefly described in the memory management section, all GPU operations in the solver’s code occur in the file `hps_parallel_leaf_ops.py`, using the described two-layer scheduling technique to compute DtN assemblies, body load reductions, and leaf interior solves without risking memory errors. The Python package PyTorch is used to organize GPU data (on tensors) and apply batched linear algebra routines to inner chunks of data.

For the DtN assembly and sparse matrix formulation, once all DtNs are assembled and transferred to a CPU tensor they are stacked into a block-diagonal matrix that is converted into a CSR sparse format. With this matrix we use indexing arrays derived from our discretization to extract A_{CC} , A_{CX} , A_{XC} , and A_{XX} , of which A_{CC} is most important as it represents the statically-condensed sparse system $\mathbf{T}$. The factorization of $\mathbf{T}$ and its use as a solver is CPU only due to the currently supported sparse direct solvers (MUMPS, SuperLU). Currently no data is permanently stored on the GPU - once DtN maps, reduced body loads, or interior leaf solutions are computed, they are transferred back to the CPU for their necessary use (building $\mathbf{T}$, forming the right-hand-side in factorized solves for leaf boundaries, and the final result, respectively).

REFERENCES

Kump, J., Yesypenko, A., and Martinsson, P.-G. (2025). A two-level direct solver for the hierarchical poincaré-steklov method. *arXiv preprint arXiv:2503.04033*.

Table 1. Command-line arguments accepted by `argparse_driver`. Required arguments are marked with (*); all others are optional. Flag arguments take no value.

Argument	Description
<i>Discretization</i>	
<code>--d</code>	Spatial dimension of the problem (2 or 3). Default: 2.
<code>--p(*)</code>	Chebyshev polynomial degree, applied uniformly in all dimensions.
<code>--p0, --p1, --p2</code>	Chebyshev degree in the x_0, x_1, x_2 directions (overrides <code>--p</code> , <code>--p2</code> is 3D only).
<code>--n(*)</code>	Number of Chebyshev points per dimension, applied uniformly.
<code>--n0, --n1, --n2</code>	Points in the x_0, x_1, x_2 directions (overrides <code>--n</code> , <code>--n2</code> is 3D only).
<i>Domain and PDE</i>	
<code>--pde(*)</code>	PDE identifier string (e.g. <code>poisson</code> , <code>bfield.constant</code>).
<code>--domain(*)</code>	Domain geometry string (e.g. <code>square</code> , <code>annulus</code> , <code>curved</code>).
<code>--box_xlim</code>	Upper limit of the domain in x_0 . Default: 1.0.
<code>--box_ylim</code>	Upper limit of the domain in x_1 . Default: 1.0.
<code>--box_zlim</code>	Upper limit of the domain in x_2 (3D only). Default: 1.0.
<i>Boundary conditions and wave parameters</i>	
<code>--bc(*)</code>	Boundary condition identifier (e.g. <code>free_space</code> , <code>log_dist</code> , <code>pulse</code>).
<code>--ppw</code>	Points per wavelength; used to derive the wavenumber kh for oscillatory problems.
<code>--nwaves</code>	Number of waves in domain; alternative to <code>--ppw</code> for setting kh .
<code>--kh</code>	Wavenumber kh set directly; mutually exclusive with <code>--ppw</code> and <code>--nwaves</code> .
<code>--periodic_bc</code>	Flag. Apply periodic boundary conditions in the x_0 direction.
<i>Time stepping (parabolic problems)</i>	
<code>--delta_t</code>	Time step size Δt for parabolic (heat/convection-diffusion) problems.
<code>--num_timesteps</code>	Number of time steps to advance the solution.
<i>Solver and assembly</i>	
<code>--solver</code>	Direct solver backend (e.g. <code>mumps</code> , <code>superlu</code>).
<code>--sparse_assembly</code>	Sparse matrix assembly strategy. Default: <code>reduced_gpu</code> ; falls back to <code>reduced_cpu</code> if CUDA is unavailable.
<code>--no_condense</code>	Flag. Skip static condensation; assemble and solve the full (uncondensed) system.
<i>Output and diagnostics</i>	
<code>--pickle</code>	Path to a file where the result dictionary will be serialised with <code>pickle</code> .
<code>--store_sol</code>	Flag. Add the solution and grid coordinates to the pickle output.
<code>--disable_cuda</code>	Flag. Force CPU execution by hiding all CUDA devices.
<code>--test_components</code>	Run component-level tests before solving. Default: <code>False</code> .
<code>--visualize</code>	Render a 3D visualisation of the solution after solving. Default: <code>False</code> .